

Chapter 6: Support Vector Machine

2023

Joseph Casey

UAB



Table of Contents

- 1 Separating Hyperplanes
- 2 Lagrangian and KKT Conditions
- 3 Support Vector Machine: Hard Margin
- 4 Support Vector Machine: Soft Margin
- 5 Kernels

Table of Contents

1 Separating Hyperplanes

2 Lagrangian and KKT Conditions

3 Support Vector Machine: Hard Margin

4 Support Vector Machine: Soft Margin

5 Kernels



Background and Setting

- We have seen multiple classification algorithms in Chapter 4 including: **Linear Regression**, **Logistic Regression**, and **LDA**. All of these assume a linear boundary prior to a possible transformation.
- While LDA and Logistic Regression are usually understood through a probabilistic lens, SVM is usually understood from a geometric one.
- for **SVM**, let's state the following:

Idea: Model the boundary between two classes as linear with the best generalizability to new data.

Goal: Classify $Y \in \{-1, 1\}$ based on $\mathbf{x} \in R^p$.

Assume: Our data points are linearly separable and our decision function is linear of the form:

$$f(\mathbf{x}) = \beta^T \mathbf{x} + \beta_0 \quad (1)$$

- $f(\mathbf{x}) = 0$ will define a hyperplane decision boundary that we can use for classification.

Hyperplane Equation

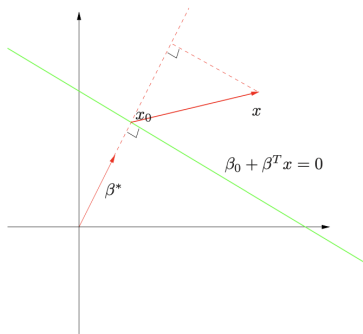


Figure 1: Hastie et al. 2009

- The equation of our hyperplane, or affine set is given by $f(\mathbf{x}) = \beta^T \mathbf{x} + \beta_0 = 0$.
- $\beta^* = \frac{\beta}{\|\beta\|}$ is the unit vector normal to the plane. (notice: $\beta^T(\mathbf{x}_1 - \mathbf{x}_2) = 0$ for any two points on the hyperplane)
- For any point $\mathbf{x}_0$ on the hyperplane, $\beta^T \mathbf{x}_0 = -\beta_0$.
- A point $\mathbf{x}$ in the plane in the direction of β satisfies the equation $f(\mathbf{x}) > 0$, and a point in the plane in the opposite direction satisfies $f(\mathbf{x}) < 0$.

- The signed distance of any point $\mathbf{x}$ to the hyperplane $f(\mathbf{x}) = 0$ is given by:

$$\beta^{*T}(\mathbf{x} - \mathbf{x}_0) = \frac{1}{\|\beta\|}(\beta^T \mathbf{x} + \beta_0) = \frac{1}{\|f'(\mathbf{x})\|} f(\mathbf{x})$$

Perceptron

- Assume we have a dataset: $(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N)$ where $y_i \in \{1, -1\}$ and $x_i \in \mathbb{R}^p$.
- Consider the decision function: $f(\mathbf{x}) = \beta^T \mathbf{x} + \beta_0$. If $f(\mathbf{x}_i) < 0$, then we predict $y_i = -1$ and if $f(\mathbf{x}) > 0$ then we predict $y_i = 1$.
- Consider the following quantity:

$$D(\beta, \beta_0) = - \sum_{i \in M} y_i (\beta^T x_i + \beta_0) \quad (2)$$

where M is the set of misclassified points. We look to minimize D with respect to β, β_0 (The perceptron algorithm uses stochastic gradient descent).

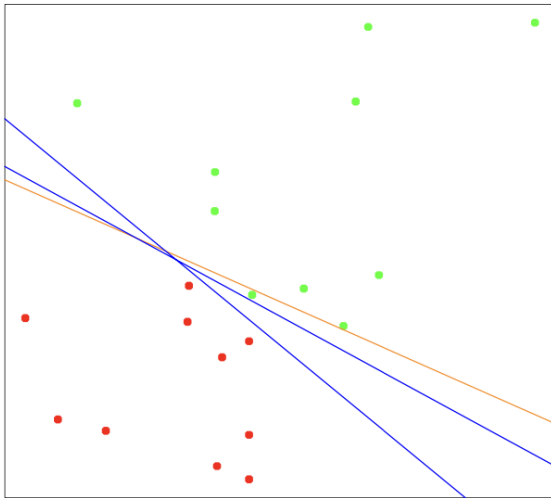


Figure 2: Separating Hyperplanes: Linear/Logistic Regression and Perceptron for binary classification. Hastie et al. 2009

Maximal Separating Hyperplane

- Recall at the start, the idea behind SVM will be to classify with the best generalizability to new data: Maximal Separating Hyperplane
- A Maximal Separating Hyperplane is a hyperplane that maximizes the margin to the data points of each of our two classes.

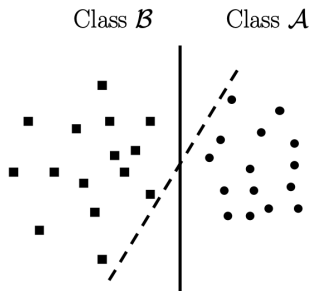


Figure 3: Solid line decision boundary vs. Dotted line decision boundary

Maximally Separating Hyperplane

Combining the technique of the Perceptron with the idea to maximize the margin, we get the following optimization problem:

$$\begin{aligned}
 & \max_{\beta, \beta_0, \|\beta\|=1} M \\
 & \text{subject to } y_i(\beta^T x_i + \beta_0) \geq M, \\
 & \quad \forall i = 1, \dots, N
 \end{aligned} \tag{3}$$

We can replace these conditions with:

$$\begin{aligned}
 & \max_{\beta, \beta_0} M \\
 & \text{subject to } \frac{1}{\|\beta\|} y_i(\beta^T x_i + \beta_0) \geq M
 \end{aligned} \tag{4}$$

by normalizing β in the constraint rather than the object function.

Maximal Separating Hyperplane

We can then set $\|\beta\| = \frac{1}{M}$ and our conditions finally become:

$$\begin{aligned} \min_{\beta, \beta_0} & \frac{1}{2} \|\beta\|^2 \\ \text{subject to } & y_i(\beta^T x_i + \beta_0) \geq 1, i = 1, \dots, N \end{aligned} \quad (5)$$

Because of how we chose β , the margin which we look to maximize can be given by $\frac{1}{\|\beta\|}$. Hence maximizing the margin is equivalent to minimizing $\|\beta\|$.

- This is called a **constrained optimization** problem, and it can be shown that it defines a **convex** optimization problem.
- This will be the optimization problem we look to solve for SVM.

Separation with margin

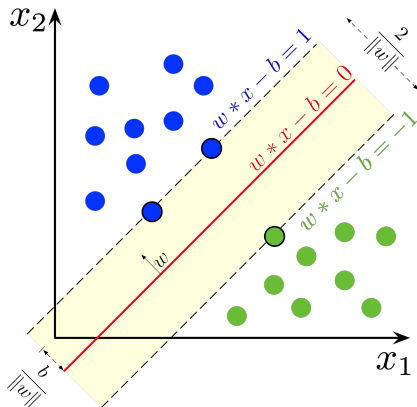


Figure 4: Hyperplane maximizing margin, (cite: https://en.m.wikipedia.org/wiki/File:SVM_margin.png)

Table of Contents

1 Separating Hyperplanes

2 Lagrangian and KKT Conditions

3 Support Vector Machine: Hard Margin

4 Support Vector Machine: Soft Margin

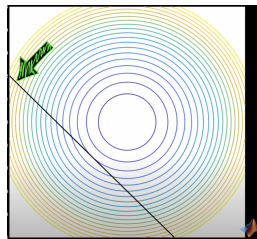
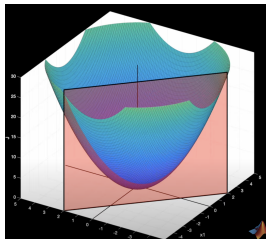
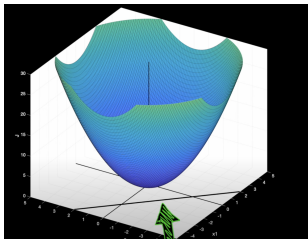
5 Kernels

Equality Constrained Optimization

Consider the following (convex) equality constrained optimization problem:

$$\begin{aligned} \min f(x) \\ \text{subject to } g(x) = 0 \end{aligned}$$

(6)



Equality Constrained Optimization

- Solution(s) will be x satisfying:

$$\nabla f(x) = \lambda \nabla g(x) \quad (7)$$

This becomes:

$$\nabla(f(x) - \lambda g(x)) = 0 \quad (8)$$

or more generally for multiple equality constraints $g_i(x) = 0$, $i = 1, \dots, m$:

$$\nabla(f(x) - \sum_{i=1}^m \lambda_i g_i(x)) = 0 \quad (9)$$

(proof: <https://math.libretexts.org/Bookshelves/Calculus>)

- Let's call $f(x) - \sum_{i=1}^m \lambda_i g_i(x)$ the **Lagrangian** for the optimization problem and denote it as:

$$L(x, \lambda) = f(x) - \sum_{i=1}^m \lambda_i g_i(x) \quad (10)$$



Equality Constrained Optimization

- Thus to solve a linearly constrained optimization problem, we first write the form of the corresponding Lagrangian, and then set $\nabla L(x, \lambda) = 0$
- To determine whether the stationary points found are minimums or maximums, we would need to consider the Bordered Hessian matrix of our Lagrangian. (see: https://en.wikipedia.org/wiki/Hessian_matrix - "Bordered Hessian")

Inequality Constrained Optimization

Let's consider a similar problem to above but with an inequality constraint instead:

$$\begin{aligned} \min f(x) \\ \text{subject to } g(x) \leq 0 \end{aligned} \quad (11)$$

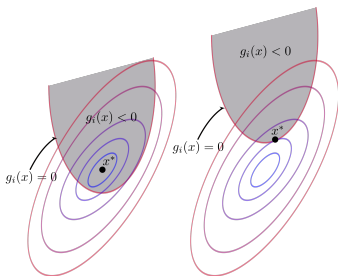
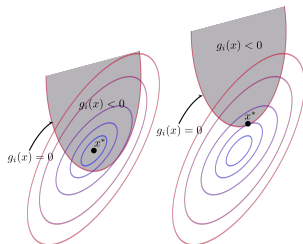


Figure 5: x^* represents the unconstrained minimum of $f(x)$ on the left, and the constrained minimum on the right

- On the left we have two diagrams: the left diagram represents the case where our inequality constraint case is equivalent to the unconstrained case of minimizing $f(x)$.
- On the right, the minimum of the unconstrained $f(x)$ is not in the constrained region, and the constrained minimum is on the boundary: $g(x) = 0$

Inequality Constrained Optimization



- Consider the diagram on the right. Since the optimal solution x^* is not at the unconstrained minimum of $f(x)$, there exists a descent direction p such that:

$$\nabla f(x^*)p < 0 \quad (12)$$

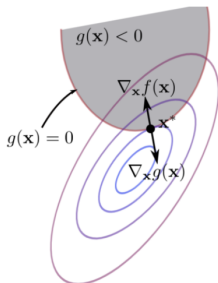
(Since $g(x^*) = 0$, x^* is a maximum for $g(x)$).

- For any descent direction p satisfying (12), we have:

$$\nabla g(x^*)p \geq 0 \quad (13)$$

- Combining (12) and (13) tells us $\text{sign} \nabla f(x^*) = -\text{sign} \nabla g(x^*)$ which means the gradients of our object function and constraint are parallel at the optimal value and in opposite directions

Inequality Constrained Optimization



- At the optimal point, we have:

$$\begin{aligned}\nabla f(x) &= -\lambda \nabla g(x) \\ \lambda &\geq 0\end{aligned}\quad (14)$$

- Thus our Lagrangian becomes:

$$\begin{aligned}L(x, \lambda) &= f(x) + \lambda g(x) \\ \lambda &\geq 0 \\ \lambda g(x) &= 0\end{aligned}\quad (15)$$

- The last condition handles both of the diagrams from above: If the unconstrained minimum x^* is in the constrained region, then $g(x) < 0$ and we have $\lambda = 0$ giving us back the unconstrained optimization problem. $\lambda > 0$ only when $g(x) = 0$.

Inequality Constrained Optimization

- For m inequality constraints, $g_i(x) \leq 0$ we have our Lagrangian to be:

$$L(x, \lambda) = f(x) + \sum_{i=1}^m \lambda_i g_i(x) \quad (16)$$

where $\lambda_i \geq 0$. Note that for the equality constrained problem, we did not have a sign requirement for our lambdas.

- Thus we want to minimize this Lagrangian for the inequality constrained optimization problem, subject the following so-called "KKT conditions":

KKT Conditions

The following conditions summarize what we required above in order to have an optimal solution, (x^*, λ^*) , to the inequality constrained optimization problem:

$$\begin{aligned} \min f(x) \\ \text{subject to } g_i(x) \leq 0, \quad i = 1, \dots, m \end{aligned} \tag{17}$$

- Condition 1 (Primal Feasibility): $g_i(x^*) \leq 0$
- Condition 2 (Stationarity): $\nabla f(x^*) + \sum_{i=1}^m \lambda_i^* \nabla g_i(x^*) = 0$
- Condition 3 (Dual Feasibility): $\lambda^* \geq 0$
- Condition 4 (Complimentary Slackness): $\sum_{i=1}^m \lambda_i^* g_i(x^*) = 0$

(We will see that these conditions also give us a nice way to solve the optimization problem for SVM)

Table of Contents

- 1 Separating Hyperplanes
- 2 Lagrangian and KKT Conditions
- 3 Support Vector Machine: Hard Margin**
- 4 Support Vector Machine: Soft Margin
- 5 Kernels

SVM: Hard Margin

- Recall our optimization problem for SVM:

$$\min_{\beta, \beta_0} \frac{1}{2} \|\beta\|^2 \quad (18)$$

subject to $y_i(\beta^T x_i + \beta_0) \geq 1, i = 1, \dots, N$

- Consider the Lagrangian for the optimization problem:

$$L_p(\beta, \beta_0, \alpha) = f(\beta) + \sum_{i=1}^N \alpha_i g_i(\beta) = \quad (19)$$

$$\frac{1}{2} \|\beta\|^2 + \sum_{i=1}^N \alpha_i [1 - y_i(\beta^T x_i + \beta_0)] \quad (20)$$

KKT Theorem

(β^*, α^*) are an optimal solution to (18) iff the KKT conditions are satisfied.

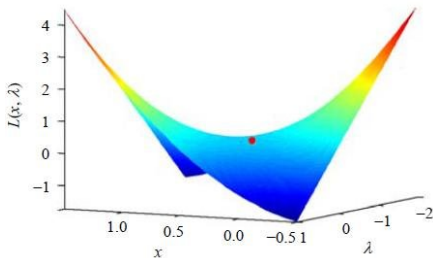


Figure 6: (cite: researchgate.net)

SVM: Hard Margin

- To solve this, we should minimize the corresponding Lagrangian with respect to β, β_0 subject to $\alpha_i \geq 0$:

$$\begin{aligned} \min_{\beta, \beta_0} L_p(\beta, \beta_0) &= \min_{\beta, \beta_0} f(\beta) + \sum_{i=1}^N \alpha_i g_i(\beta) = \\ \min_{\beta, \beta_0} \frac{1}{2} \|\beta\|^2 - \sum_{i=1}^N \alpha_i [y_i(\beta^T x_i + \beta_0) - 1] \end{aligned} \quad (21)$$

- Setting the gradient to 0 ($\nabla L(\beta, \beta_0) = 0$), gives the following results:

$$\frac{\partial L}{\partial \beta_0} = \sum_{i=1}^N \alpha_i y_i = 0 \quad (22)$$

$$\text{and } \frac{\partial L}{\partial \beta} = 0 \implies$$

$$\beta = \sum_{i=1}^N \alpha_i y_i x_i \quad (23)$$

SVM: Hard Margin

- Since our stationary point is a saddle point, we want to solve:

$$\max_{\alpha} \min_{\beta, \beta_0} L(\beta, \beta_0, \alpha) \quad (24)$$

Plugging in our previous results, this becomes:

$$\begin{aligned} \max_{\alpha} L_d = \\ \max_{\alpha} \sum_{i=1}^N \alpha_i - \frac{1}{2} \sum_{i=1}^N \sum_{k=1}^N \alpha_i \alpha_k y_i y_k x_i^T x_k \\ \text{subject to } \alpha_i \geq 0 \\ \sum_{i=1}^N \alpha_i y_i = 0 \end{aligned} \quad (25)$$

To satisfy the KKT conditions, we must also have $\alpha_i [1 - y_i(\beta^T x_i + \beta_0)] = 0$

SVM: Hard Margin

- Naively, we can set the derivative of L_d with respect to α_i 's equal to 0 to solve. Solving this maximization problem will give us the optimal α_i 's which will allow us in-turn to solve for β .
- Due to the KKT conditions, most of the α_i 's will be zero. So:

$$\beta = \sum_{i=1}^N \alpha_i y_i x_i = \sum_{x_i \in \mathcal{SV}} \alpha_i y_i x_i \quad (26)$$

- Our solution depends only on the points that lie exactly on the margin. These points are called **support vectors**.
- The support vectors satisfy the equation:

$$y_i(\beta^T x_i + \beta_0) = 1 \text{ or } \beta_0 = y_i - \beta^T x_i \quad (27)$$

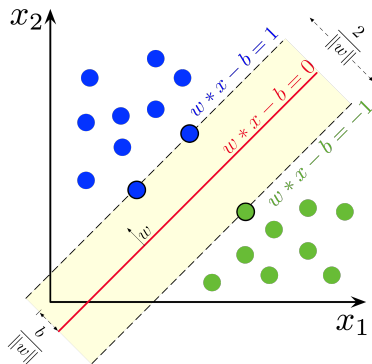
- For numerical stability, we solve for β_0 using the average of all of the support vectors:

$$\beta_0 = \frac{1}{|\mathcal{SV}|} \sum_{x_i \in \mathcal{SV}} (y_i - \beta^T x_i) \quad (28)$$

SVM: Hard Margin

Having β and β_0 then gives us our decision function:

$$f(x) = \beta^T x + \beta_0 \quad (29)$$



- Given a new data point x , we classify it as C_1 if $f(x) > 0$ and C_2 if $f(x) < 0$. Even though we train using the margin, we don't consider the margin when classifying new points.

Table of Contents

- 1 Separating Hyperplanes
- 2 Lagrangian and KKT Conditions
- 3 Support Vector Machine: Hard Margin
- 4 Support Vector Machine: Soft Margin**
- 5 Kernels

SVM: Soft Margin

- For the Hard Margin case, we supposed our data were linearly separable. In practice however, this usually doesn't happen. Let us relax this assumption and see how this effects the optimization problem
- Assume now that our data are not linearly separable.

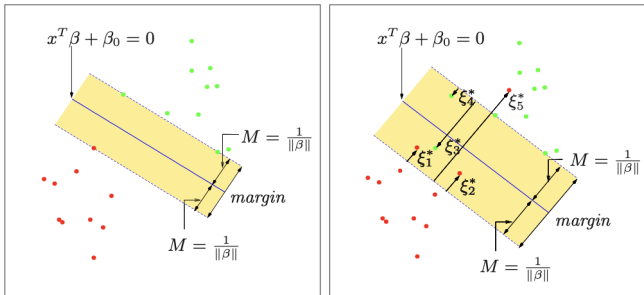


Figure 7: Left - Hard Margin, Right - Soft Margin

SVM: Soft Margin

- We still start by defining so-called "slack variables": $\xi = (\xi_1, \xi_2, \dots, \xi_N)$ corresponding to each of our data points $(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N)$.
- We then modify our hard margin optimization problem to be:

$$\begin{aligned} \min_{\beta} \quad & \frac{1}{2} \|\beta\|^2 \\ \text{subject to} \quad & y_i(\beta^T x_i + \beta_0) \geq 1 - \xi_i, \\ & \xi_i \geq 0, \forall i = 1, \dots, N, \end{aligned} \tag{30}$$

- Notice however that this constraint would have no effect on the optimization problem. We would just let $\beta \rightarrow 0$ and let ξ_i be 0 for things classified correctly beyond the margin, and whatever necessary to correct misclassified points, or points too close to the margin.
- Instead, let's limit the size of $\sum_{i=1}^N \xi_i$ by treating it as a penalty term of our object function.

SVM: Soft Margin

Thus, we should consider the optimization problem:

$$\begin{aligned}
 & \min_{\beta} \frac{1}{2} \|\beta\|^2 + C \sum_{i=1}^N \xi_i \\
 & \text{subject to } y_i(\beta^T x_i + \beta_0) \geq 1 - \xi_i, \\
 & \xi_i \geq 0, \forall i = 1, \dots, N
 \end{aligned} \tag{31}$$

- So our $\sum_{i=1}^N \xi_i$ is a penalty term controlled by the hyperparameter $C > 0$. The bigger C , the more we penalize on having $\xi_i > 0$. If $C = \infty$, we would penalize an infinite amount on having any $\xi_i > 0$, giving us the hard margin case. When C is small, we can have large ξ_i or many x_i violating the margin.

SVM: Soft Margin

- The corresponding Lagrangian is:

$$L_p = \frac{1}{2} \|\beta\|^2 + C \sum_{i=1}^N \xi_i - \sum_{i=1}^N \alpha_i [y_i(\beta^T x_i + \beta_0) - (1 - \xi_i)] - \sum_{i=1}^N \mu_i \xi_i \quad (32)$$

- Setting the derivatives w.r.t $\beta, \beta_0, \xi_i = 0$ gives:

$$\begin{aligned} \beta &= \sum_{i=1}^N \alpha_i y_i x_i \\ \sum_{i=1}^N \alpha_i y_i &= 0 \end{aligned} \quad (33)$$

$$\alpha_i = C - \mu_i$$

where $\alpha_i, \mu_i, \xi_i \geq 0, \forall i = 1, \dots, N$

SVM: Soft Margin

- Plugging these back into L_p , gives our optimization problem:

$$\max_{\alpha} L_d = \sum_{i=1}^N \alpha_i - \frac{1}{2} \sum_{i=1}^N \sum_{k=1}^N \alpha_i \alpha_k y_i y_k x_i^T x_k \quad (34)$$

subject to:

$$\begin{aligned} \sum_{i=1}^N \alpha_i y_i &= 0, \\ 0 &\leq \alpha_i \leq C \end{aligned} \quad (35)$$

along with the KKT conditions:

$$\begin{aligned} \alpha_i [y_i (\beta^T x_i + \beta_0) - (1 - \xi_i)] &= 0 \\ \mu_i \xi_i &= 0 \end{aligned} \quad (36)$$

SVM: Soft Margin

- Similar to the hard margin case, after solving for our optimal α_i 's, we will have:

$$\beta = \sum_{x_i \in \mathcal{SV}} \alpha_i y_i x_i \quad (37)$$

depending only on the support vectors. Here support vectors refers three kinds of points: those on the margin boundary ($\xi_i = 0$), those within the margin on the correct side ($0 \leq \xi_i \leq 1$), and those on the wrong side ($\xi_i > 1$).

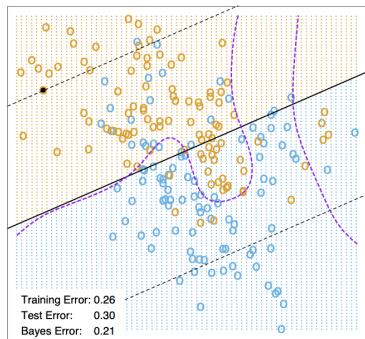
- We can calculate our β_0 using any support vector in the margin, but instead we use an average of all of them for numerical stability:

$$\beta_0 = \frac{1}{|\mathcal{SV}|} \sum_{x_i \in \mathcal{SV}} (y_i - \beta^T x_i) \quad (38)$$

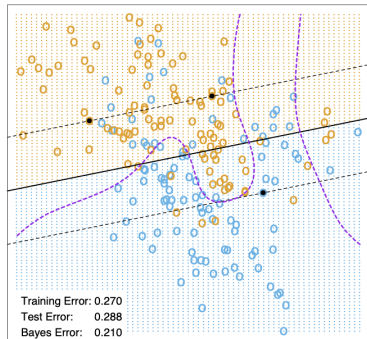
- This gives us a decision function: $f(x) = \beta^T x + \beta_0$

SVM: Soft Margin

- For hard margin SVM, we didn't have any hyperparameters to specify, but for soft margin we need to specify the parameter C beforehand. The bigger C , the fewer support vectors and vice versa.



$C = 0.01$



$C = 10000$

Table of Contents

- 1 Separating Hyperplanes
- 2 Lagrangian and KKT Conditions
- 3 Support Vector Machine: Hard Margin
- 4 Support Vector Machine: Soft Margin
- 5 Kernels

Non-linear Decision Boundary

- Sometimes a linear decision boundary doesn't suit the data, even with the allowance of misclassification. To handle this, we can transform our feature space to a higher dimension with the understanding that linearly inseparable data can be linearly separable in a higher dimension.

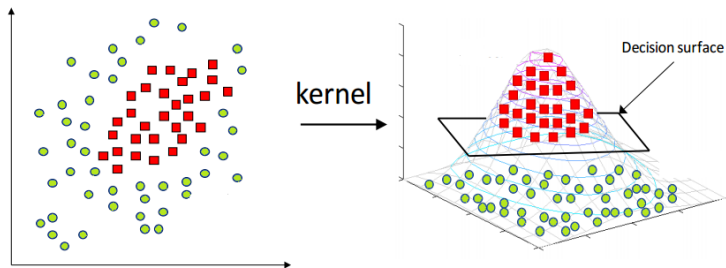


Figure 8: (cite: <https://medium.com/@apurvjain37/support-vector-machine-s-v-m-classifiers-and-kernels-9e13176c9396>)

Things to Consider

- Mapping data features to a higher (even infinite) dimension in general can have a huge effect on computation cost. This is especially true for SVM.
- To help with this, we can take a closer look at the optimization problem, and consider only special kinds of feature transformations.

Recall, we want to solve the following:

$$\max_{\alpha} L_d = \max_{\alpha} \sum_{i=1}^N \alpha_i - \frac{1}{2} \sum_{i=1}^N \sum_{k=1}^N \alpha_i \alpha_k y_i y_k x_i^T x_k \quad (39)$$

- We see a special dependence of this function on the inner product of our data.
- Similarly, our decision function depends only on the inner product:

$$f(x) = \beta^T x + \beta_0 = \sum_{x_i \in \mathcal{SV}} \alpha_i y_i x_i^T x + \beta_0 \quad (40)$$

Transforming Data

- We now consider our feature vectors transformed by some function h . Our data are given by x_i , then h is defined by: $h(x_i)$, where $h(x_i) = (h_1(x_i), h_2(x_i), \dots, h_m(x_i))$. Then rewriting our optimization problem using inner products, the Lagrangian becomes:

$$L_D = \sum_{i=1}^N \alpha_i - \frac{1}{2} \sum_{i=1}^N \sum_{k=1}^N \alpha_i \alpha_k y_i y_k \langle h(x_i), h(x_k) \rangle. \quad (41)$$

- Our solution is of the form:

$$f(x) = \beta^T h(x) + \beta_0 = \sum_{i=1}^N \alpha_i y_i \langle h(x), h(x_i) \rangle + \beta_0 \quad (42)$$

- We can define the kernel as:

$$K(x, x_k) = \langle h(x), h(x_k) \rangle \quad (43)$$

Kernels

- We now have our optimization problem and decision function written in terms of the kernel:

$$L_D = \sum_{i=1}^N \alpha_i - \frac{1}{2} \sum_{i=1}^N \sum_{k=1}^N \alpha_i \alpha_k y_i y_k K(x_i, x_k) \quad (44)$$

$$f(x) = \beta^T h(x) + \beta_0 = \sum_{i=1}^N \alpha_i y_i K(x, x_i) + \beta_0$$

- We will see that for appropriately chosen kernels, we can avoid the direction calculations involving the high-dimensionally transformed feature vectors.

(SVM Kernels: <https://www.youtube.com/watch?v=XUj5JbQihlU&t=1553s>)

Kernel Examples

Kernel Types

degree d polynomial:

$$K(x, x') = (1 + \langle x, x' \rangle)^d$$

radial:

$$K(x, x') = \exp(-\gamma \|x - x'\|^2)$$

neural network:

$$K(x, x') = \tanh(\kappa_1 \langle x, x' \rangle + \kappa_2)$$

Example Consider a feature space consisting of x_1 and x_2 . Consider the degree 2 polynomial kernel:

- $K(x, x') = (1 + \langle x, x' \rangle)^d = (1 + x_1 x'_1 + x_2 x'_2)^2 = 1 + 2x_1 x'_1 + 2x_2 x'_2 + (x_1 x'_1)^2 + (x_2 x'_2)^2 + 2x_1 x'_1 x_2 x'_2$
- Thus, if we define $h(x) = (h_1(x), h_2(x), \dots, h_6(x)) = (1, \sqrt{2}x_1, \sqrt{2}x_2, x_1^2, x_2^2, \sqrt{2}x_1 x_2)$
- $K(x, x') = \langle h(x), h(x') \rangle$ and our solution can be written as $\hat{f}(x) = \sum_{i=1}^N \hat{\alpha}_i y_i K(x, x_i) + \hat{\beta}_0$.

SVM Kernel Examples

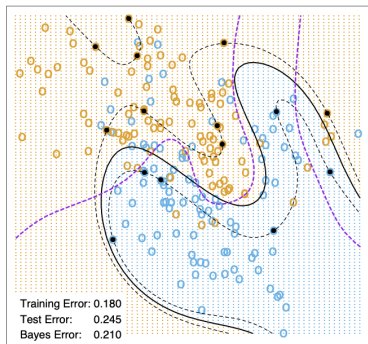


Figure 9: SVM with degree 4 polynomial kernel. Hyperparameter C chosen to be best. Hastie et al. 2009

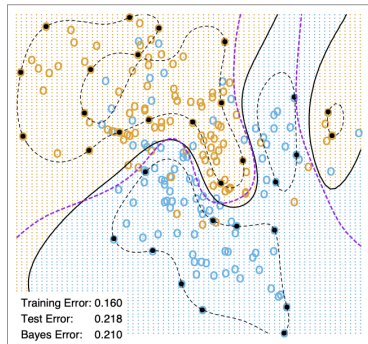


Figure 10: SVM with radial kernel. Hyperparameter C chosen to be best. Hastie et al. 2009

References I



Hastie, T. et al. (2009). *The elements of statistical learning: data mining, inference, and prediction*. Vol. 2. Springer.